

React & Optimization Techniques

A clear, beginner-friendly visual guide

1. React's Component Tree — How React Works Behind the Scenes

React builds a **tree of components** — just like a family tree. The top-most component is the **root**, and everything else branches out from it. When something changes (state or props), React re-renders the affected components.

Key ideas:

- React keeps a **Virtual DOM** — a lightweight copy of the real browser DOM.
- When state changes, React re-runs the component function and compares old vs new output.
- Only the **changed parts** are updated in the real DOM — this is called **Reconciliation**.
- Child components re-render when their parent re-renders (unless optimized).

App (Root)
Header Main Footer
NavBar Logo Sidebar Content Links

2. Analyzing Component Executions via React DevTools Profiler

The **React DevTools Profiler** is a browser extension that helps you see **which components re-rendered**, **how long they took**, and **why they re-rendered**. It's your best tool for finding performance problems.

Step	What to do
1	Install React DevTools extension in Chrome or Firefox
2	Open browser DevTools → click the "Profiler" tab
3	Click the record button (circle), interact with your app, then click stop
4	Look at the flame graph — wide bars = slow components
5	Check 'Why did this render?' to find unnecessary re-renders

3. Avoiding Re-renders with memo()

React.memo() is a wrapper that tells React: "Only re-render this component if its props actually changed." If the parent re-renders but passes the same props, the child is **skipped** — saving time.

Without memo:

Parent re-renders

↓

Child re-renders too
(even if props didn't change)

■ Wasted work

With memo:

Parent re-renders

↓

React checks: props same?

YES → Child is **skipped!**

■ Efficient

How to use it:

```
import { memo } from 'react';

const MyComponent = memo(function MyComponent({ name }) {
  return Hello, {name}!;
});

// Now MyComponent only re-renders when 'name' changes
```

***Tip:** Don't wrap every component with memo — only use it when a component re-renders often and its props rarely change. Overusing memo can slow things down.*

4. Avoiding Re-renders with Clever Structuring

Sometimes you don't even need memo. By **restructuring your components smartly**, you can prevent unnecessary re-renders naturally. The idea: **move state closer to where it's used**.

Example — The Problem:

```
function App() {
  const [count, setCount] = useState(0);
  return (
    /* Re-renders whenever count changes! */
    setCount(c => c + 1)>Click
    {count}
  );
}
```

Fix — Extract the stateful part:

```
function Counter() { // State lives here now
  const [count, setCount] = useState(0);
```

```

return (
  <>
    setCount(c => c + 1)}>Click
  {count}
);
}

function App() {
  return (
    /* No longer re-renders on count change! */
  );
}

```

5. The useCallback() Hook

Every time a component re-renders, **functions defined inside it are recreated** as brand new objects. This can break memo() because the child sees a 'new' function prop each time. **useCallback** fixes this by **remembering the same function** between renders.

■ Problem:

Each render creates a new handleClick function.

Even if wrapped in memo, child re-renders because function reference changed.

■ Solution:

useCallback caches the function between renders.

Child wrapped in memo sees the SAME function → skips re-render.

Syntax:

```

const handleClick = useCallback(() => {
  doSomething(userId);
}, [userId]); // Recreated only when userId changes
// Pass to child safely – child won't re-render unnecessarily

```

6. The useMemo() Hook

useMemo is used to **cache the result of an expensive calculation**. Instead of recalculating every render, React remembers the result and only recalculates when the inputs (dependencies) change.

	useCallback	useMemo
Caches	A function	A computed value / result
Returns	The same function	The same value
Use when	Passing callbacks to children	Expensive calculations
Example	handleClick, fetchData	filtered list, sorted array

Example — avoid recalculating a filtered list:

```
const expensiveResult = useMemo(() => {  
  return items.filter(item => item.price > 100); // Only runs when 'items' changes  
}, [items]);
```

***Tip:** Don't use useMemo for simple operations like adding two numbers. Use it only when a calculation is truly expensive (filtering large lists, sorting, heavy math).*

7. Why Keys Matter When Managing State

React uses **keys** to **identify each item in a list**. When the key changes, React thinks it's a completely **different component** and throws away the old one (including its state) and creates a fresh one.

Scenario	What React Does
Same key, same position	Keeps existing component, updates props only
Key changes	Destroys old component + creates brand new one
Item removed from list	React destroys that component
Items reordered	React matches by key, not by position

Always use stable, unique keys from your data (like IDs), never use array index as key in dynamic lists:

```
// Bad - using index as key  
{items.map((item, index) => )}  
  
// Good - using unique ID from data
```

```
{items.map(item => )}
```

Using Keys to Reset a Component

You can **intentionally change a key** to force React to throw away a component and start fresh. This is useful when you want to reset a component's internal state (like a form) without writing extra code.

```
// Change 'selectedUser' to force ProfileForm to reset completely  
  
// When selectedUser.id changes, the old form is destroyed  
// and a fresh one is created – state is reset automatically!
```

8. State Scheduling & Batching

When you call **setState**, React doesn't update the screen *immediately*. Instead, it **schedules** the update and may **batch** multiple updates together into a single re-render — making your app faster.

■ State Scheduling

setState() doesn't update state immediately. React queues the update and processes it at the right time (end of event, etc.).

This is why reading state right after setState gives the OLD value.

■ State Batching

If you call setState multiple times in one event handler, React batches them into ONE re-render.

React 18+ batches even inside setTimeout and async functions (automatic batching).

Example of batching:

```
function handleClick() {  
  setCount(c => c + 1); // scheduled  
  setName('Alice'); // scheduled  
  setActive(true); // scheduled  
  // React re-renders ONCE with all three updates – not three times!  
}
```

Note: Always use the **updater function form** (setCount(c => c + 1)) when new state depends on old state — this ensures you get the latest value even when updates are batched.

9. Optimizing React with Million.js

Million.js is a drop-in library that makes React apps significantly faster — sometimes up to **70% faster**. It replaces React's virtual DOM diffing with a smarter, more direct approach called the **block virtual DOM**.

Feature	React Default	With Million.js
DOM Updates	Virtual DOM diffing	Block Virtual DOM (smarter)
Re-render Speed	Standard	Up to 70% faster
Setup Effort	—	Minimal — drop-in replacement
Works with	—	Existing React code, no rewrite needed
Best for	—	Lists, tables, data-heavy UIs

Quick setup:

```
# 1. Install
npm install million

# 2. Wrap your React app in vite.config.js
import million from 'million/compiler';

export default {
  plugins: [million.vite({ auto: true }), react()],
};

# 3. That's it! Million.js auto-optimizes your components
```

***When to use Million.js:** Great for apps with large lists, frequent updates, or real-time data. For simple apps, React's built-in optimizations are usually enough.*

Quick Reference Summary

Tool / Concept	What it does	When to use
React Component Tree	Organizes UI as parent-child hierarchy	Always — fundamental concept
DevTools Profiler	Shows which components re-rendered & why	Debugging performance issues
memo()	Skips re-render if props unchanged	Child renders too often
Clever Structuring	Move state closer to where it's used	Prevent unnecessary re-renders
useCallback()	Caches a function between renders	Passing callbacks to memo children
useMemo()	Caches a computed value	Expensive calculations
Keys	Identifies list items uniquely	Lists & resetting components

State Batching	Groups multiple setState calls into one	Automatic in React 18+
Million.js	Faster DOM updates (block vDOM)	Data-heavy / high-frequency UIs